

Web Development using MERN stack

Lab

FSD-322P

Faculty Name : Ms. Kavita Saxena

Name : Sameer Sharma

Roll No. : 02314802723

Semester : 6th

Group : 6FSD-1-A



Maharaja Agrasen Institute of Technology, PSP
Area,
Sector - 22, Rohini, New Delhi - 110085

MAHARAJA AGRASEN INSTITUTE OF TECHNOLOGY

VISION

To attain global excellence through education, innovation, research, and work ethics with the commitment to serve humanity.

MISION

M1. To promote diversification by adopting advancement in science, technology, management, and allied discipline through continuous learning

M2. To foster moral values in students and equip them for developing sustainable solutions to serve both national and global needs in society and industry.

M3. To digitize educational resources and process for enhanced teaching and effective learning.

M4. To cultivate an environment supporting incubation, product development, technology transfer, capacity building and entrepreneurship.

M5. To encourage faculty-student networking with alumni, industry, institutions, and other stakeholders for collective engagement.

MAHARAJA AGRASEN INSTITUTE OF TECHNOLOGY

Department of Computer Science and Engineering

VISION

"To attain global excellence through education, innovation, research, and work ethics in the field of Computer Science and engineering with the commitment to serve humanity."

MISION

M1 To lead in the advancement of computer science and engineering through globally recognized research and quality education.

M2 To prepare students for full and ethical participation in a diverse society and encourage lifelong learning.

M3 To foster development of problem solving, programming and communication skills as an integral component of the profession.

M4 To cultivate an environment supporting incubation, product development, technology transfer, capacity building and entrepreneurship in the field of computer science and engineering.

M5 To encourage faculty, student's networking with alumni, industry, institutions, and other stakeholders for collective engagement.

Rubrics for Lab Assessment:

Rubrics		10 Marks			POs and PSOs Covered	
		0 Marks	1 Marks	2 Marks	PO	PSO
R1	Is able to identify and define the objective of the given problem?	No	Partially	Completely	PO1, PO2	PSO1, PSO2
R2	Is proposed design/procedure/algorithm solves the problem?	No	Partially	Completely	PO1,PO2, PO3	PSO1, PSO2
R3	Has the understanding of the tool/programming language to implement the proposed solution?	No	Partially	Completely	PO1,PO3, PO5	PSO1, PSO2
R4	Are the result(s) verified using sufficient test data to support the conclusions?	No	Partially	Completely	PO2,PO4, PO5	PSO2
R5	Individuality of submission?	No	Partially	Completely	PO8, PO12	PSO1, PSO3

[illegible]

EXPERIMENT – 1

AIM: Taks management tool: Login/Register to the application, add daily tasks, Assign a due date of completion. Mark them as complete/incomplete and View weekly/monthly statistics of their to-dos.

THEORY:

Building a task management tool with **HTML and CSS** focuses on **Information Architecture** and **Interface Design**. Since this is a static implementation, the theory centers on how to structure a functional "User Flow" using only markup and styling.

1. The Structure (Semantic HTML)

The foundation relies on **Semantic Tags** to define the app's regions:

- **Forms:** Used for the Login, Registration, and Task Input sections.
- **Lists (/):** Used to represent the collection of tasks.
- **Data Inputs:** Utilizing <input type="date"> for deadlines and <input type="checkbox"> for task status.

2. The Layout (Flexbox & Grid)

- **CSS Flexbox:** Best for the "Task Row," aligning the checkbox, task text, and date in a single horizontal line.
- **CSS Grid:** Ideal for the **Statistics Dashboard**, allowing you to create a responsive 2x2 or 3x3 grid of "Weekly Completion" cards.

3. State Management

Without JavaScript, we use the **CSS Sibling Selector (+ or ~)** to change styles.

- **Theory:** When a checkbox is checked, the CSS targets the adjacent label to apply text-decoration: line-through and change the text color to grey, visually "completing" the task.

CODE:

1. dashboard.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Document</title>
  </head>
  <body>
    <div class="container">
      <h1>DashBoard</h1>
      <div class="task">
        <h2>Add a Task</h2>
        <textarea
          class="desc"
          name="task"
          placeholder="Enter task description.."
          rows="8"
          cols="25"
        ></textarea>
        <div class="btn">
          <input type="date" />
          <button>Add Task</button>
        </div>
      </div>

      <div class="container2">
        <div>
          <h2>Your Tasks</h2>
          <ul class="task-list">
            <li>
              <input type="checkbox" />
              <span>Technical interview</span>
              <span class="date">2026-02-05</span>
            </li>
            <li class="completed">
              <input type="checkbox" checked />
              <span>Buy groceries</span>
              <span class="date">2026-02-04</span>
            </li>
          </ul>
        </div>

        <div>
          <h2>Statistics</h2>
          <div class="stat">
            <div>
              <h4>Weekly</h4>
              <p>Completed: 6</p>
            </div>
          </div>
        </div>
      </div>
    </div>
  </body>
</html>
```

```
        <p>Pending: 2</p>
    </div>
    <div>
        <h4>Monthly</h4>
        <p>Completed: 25</p>
        <p>Pending: 3</p>
    </div>
</div>
</div>
</div>
</div>
</body>
</html>
```

OUTPUT:

DashBoard

Add a Task

Enter task description..

dd - mm - yyyy



Add Task

Your Tasks

- ☐ Technical interview 2026-02-05
- ☒ Buy groceries 2026-02-04

Statistics

Weekly

Completed: 6

Pending: 2

Monthly

Completed: 25

Pending: 3

2. login.html

CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Task Manager - Login</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

<div class="auth-container">
  <h1>Task Manager</h1>

  <input type="text" placeholder="Username"> <br> <br>
  <input type="password" placeholder="Password">

  <button>Login</button>

  <p>Don't have an account?</p>
  <button class="secondary">Register</button>
</div>

</body>
</html>
```

OUTPUT:

Task Manager

Don't have an account?

3. style.css

CODE:

```
.container{  
  display: flex;  
  flex-direction: column;  
  align-items: center;  
  height: 100vh;  
  border: solid 1px black;  
}
```

```
.task{  
  display: flex;  
  flex-direction: column;  
  align-items: center;  
  margin-top: -6px;  
}
```

```
.desc{  
  margin-bottom: 10px;  
  border-radius: 10px;  
}
```

```
.container2{  
  display: flex;  
  flex-direction: row;  
  align-items: center;  
  gap: 3rem;  
  margin-top: 2rem;  
  padding: 1rem;  
  margin-left: 8rem;  
}
```

```
.stat{  
  display: flex;  
  flex-direction: row;  
  align-items: center;  
  margin-top: 10px;  
  gap: 3rem;  
  border: solid 1px black;  
  border-radius: 8px;  
  padding: 1rem;  
  font-size: large;  
}
```

```
.task-list{  
  display: flex;  
  flex-direction: column;  
  gap: 1rem;  
  border: solid 1px black;
```

```
padding: 1.5rem;  
border-radius: 8px;  
font-size: large;  
}
```

OUTPUT:

Dashboard

Add a Task

Enter task description..

dd-mm-yyyy



Add Task

Your Tasks

- ☐ Technical interview 2026-02-05
- ☒ Buy groceries 2026-02-04

Statistics

Weekly	Monthly
Completed: 6	Completed: 25
Pending: 2	Pending: 3

VIVA QUESTIONS

1. What is the purpose of flex: 1; on the input field inside it?

Ans: flex: 1; tells the input field to grow and take up all the available remaining space in the container. This ensures the input stays large while the date picker and button only take up as much space as they need.

2. Explain the "Box Model" in CSS and how it relates to your .card or .container elements.

Ans: The Box Model consists of **Content, Padding, Border, and Margin**. For example, in .card class, the padding: 15px creates space inside the card around the text, while margin-top: 20px creates space outside the card to separate it from the heading.

3. What is the difference between an Internal, External, and Inline CSS? Which one did you use?

Ans: External: A separate .css file linked via <link> (I used this).

- **Internal:** CSS written inside <style> tags in the HTML <head>.
- **Inline:** CSS written directly inside an HTML tag using the style attribute. Using an **External** stylesheet is best practice because it keeps the code organized and allows multiple pages (Login and Dashboard) to share the same styles.

4. What are "Pseudo-classes"? Give an example.

Ans: Pseudo-classes define a special state of an element. A common example for this project would be :hover on the buttons. It allows the button color to change when the user moves their mouse over it, providing visual feedback.

5. What are the uses of and tags?

Ans: These are **Semantic Tags** specifically designed for lists. Using them tells the browser and screen readers that the items are related and part of a collection, which improves accessibility and the structural logic of the code.

Experiment-2

Aim

To develop a full-stack blogging platform that allows users to create, view, edit, and delete blog posts using modern web technologies.

Objectives

- To understand full-stack development
- To implement CRUD operations
- To build UI using ReactJS
- To develop backend using NodeJS and ExpressJS
- To store data using MongoDB
- To manage blog content dynamically

Technologies Used

- **ReactJS** – Frontend
- **NodeJS** – Backend runtime
- **ExpressJS** – API handling
- **MongoDB** – Database
- **CSS** – Styling (**Dark theme UI**)
- **Axios** – API communication

Theory

A blogging platform allows users to create and manage posts.

Frontend

Handles UI and user interaction.

Backend

Processes requests and performs CRUD operations.

Database

Stores blog data (title + content).

Styling (Different CSS Concept)

This version uses:

- Dark mode UI
- Minimal design
- Clean card layout

Working

1. User enters blog
2. Data is sent to backend
3. MongoDB stores data
4. Blogs are displayed
5. User can edit/delete

Code:

1. App.jsx (Frontend)

```
import React, { useEffect, useState } from "react";
import axios from "axios";

function App() {
  const [title, setTitle] = useState("");
  const [content, setContent] = useState("");
  const [blogs, setBlogs] = useState([]);
  const [editId, setEditId] = useState(null);

  const fetchBlogs = async () => {
    const res = await axios.get("http://localhost:5000/blogs");
    setBlogs(res.data);
  };

  useEffect(() => {
    { fetchBlogs();
    }, []);

  const handleSubmit = async () =>
  { if (editId) {
    await axios.put(`http://localhost:5000/update/${editId}`,
    { title, content
    });
    setEditId(null);
  } else {
    await axios.post("http://localhost:5000/add",
    { title, content
    });
  }
}
```

```

setTitle("");
setContent("");
fetchBlogs();
};

const editBlog = (b) =>
{ setTitle(b.title);
  setContent(b.content);
  setEditId(b._id);
};

const deleteBlog = async (id) => {
  await axios.delete(`http://localhost:5000/delete/${id}`);
  fetchBlogs();
};

return (
  <div
    style={{ background:
      "#121212", minHeight:
      "100vh", color: "white",
      padding: "20px"
    }}>
    <h1 style={{ textAlign: "center" }}>Blog Platform</h1>

    <div
      style={{ background:
        "#1e1e1e", padding:
        "20px", borderRadius:
        "10px", maxWidth:
        "500px", margin:
        "auto"
      }}>
      <input
        placeholder="Title"
        value={title}
        onChange={(e) => setTitle(e.target.value)}
        style={{ width: "100%", marginBottom: "10px" }}
      />

      <textarea
        placeholder="Content"
        value={content}
        onChange={(e) => setContent(e.target.value)}
        style={{ width: "100%" }}
      ></textarea>

      <button onClick={handleSubmit}
        style={{
          marginTop: "10px",

```

background: "#03dac6",


```

        color: "black"
      }}>
      {editId ? "Update" : "Add Blog"}
    </button>
  </div>

  <div style={{ marginTop: "20px" }}>
    {blogs.map((b) => (
      <div key={b._id}
        style={{
          background: "#1e1e1e",
          margin: "10px auto",
          padding: "15px",
          borderRadius: "10px",
          maxWidth: "500px"
        }}>
        <h3>{b.title}</h3>
        <p>{b.content}</p>

        <button onClick={() => editBlog(b)}
          style={{ marginRight: "5px" }}>
          Edit
        </button>

        <button onClick={() => deleteBlog(b._id)}
          style={{ background: "red", color: "white" }}>
          Delete
        </button>
      </div>
    ))}
  </div>
</div>
);
}

```

export default App;

2. server.js (Backend)

```

const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");

const app = express();

app.use(express.json());
app.use(cors());

mongoose.connect("mongodb://127.0.0.1:27017/blogDB",
  { useNewUrlParser: true,
    useUnifiedTopology: true,

```

```

})
.then(() => console.log("MongoDB Connected"))
.catch(err => console.log(err));

const schema = new
  mongoose.Schema({ title: String,
    content: String,
  });

const Blog = mongoose.model("Blog", schema);

// Add
app.post("/add", async (req, res) =>
  { const blog = new Blog(req.body);
    await blog.save();
    res.send("Added");
  });

// Get
app.get("/blogs", async (req, res) =>
  { const blogs = await Blog.find();
    res.json(blogs);
  });

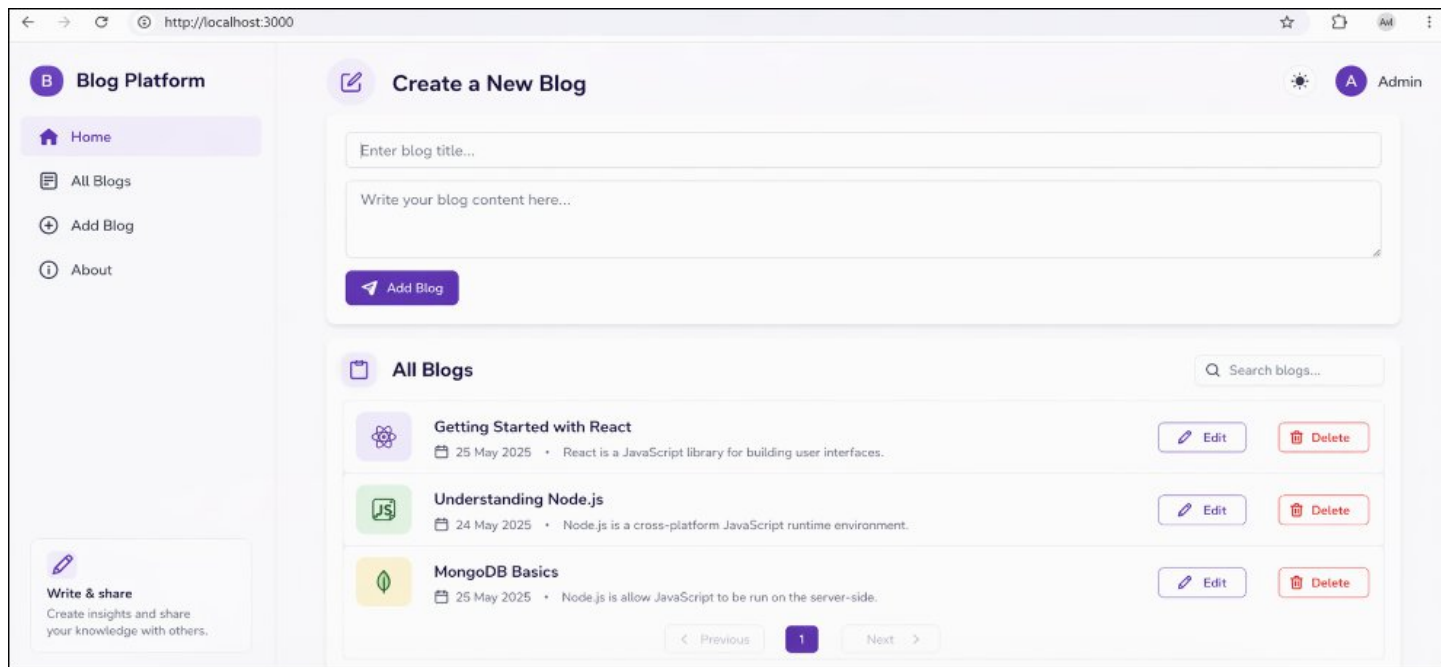
// Update
app.put("/update/:id", async (req, res) => {
  await Blog.findByIdAndUpdate(req.params.id, req.body);
  res.send("Updated");
});

// Delete
app.delete("/delete/:id", async (req, res) =>
  { await
    Blog.findByIdAndDelete(req.params.id);
    res.send("Deleted");
  });

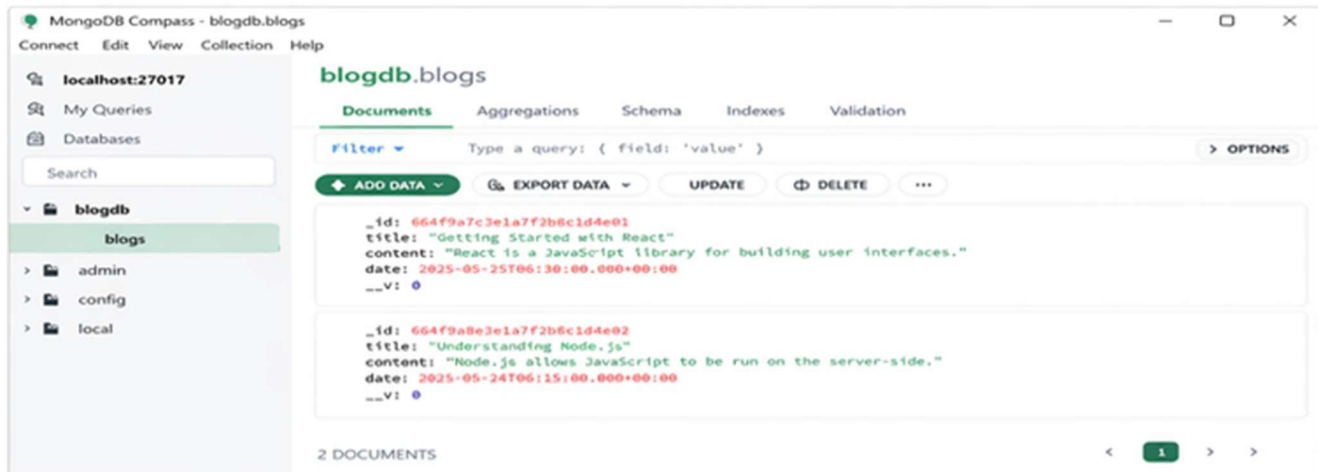
app.listen(5000, () => {
  console.log("Server running on http://localhost:5000");
});

```

Output:



MongoDB Compass (Database)



Viva Questions

1. What is a blogging platform?

A blogging platform is a software or system that allows users to create, publish, and manage blog posts and content online.

2. What is CRUD?

CRUD stands for Create, Read, Update, and Delete — the four basic operations performed on data in a database.

3. What is MongoDB used for?

MongoDB is a NoSQL database used to store and manage data in the form of documents (JSON-like format), commonly

4. What is Axios?

Axios is a JavaScript library used to make HTTP requests from the frontend to the backend, such as GET, POST, PUT, and DELETE requests for API communication.

5. What happens in edit operation?

In an edit operation, existing data is modified and updated in the database, usually by sending a PUT or PATCH request to the server.

Experiment-3

Aim

To design and develop a social media platform that allows users to create posts, like posts, and delete posts using full-stack web technologies.

Objectives

- To understand full-stack web development
- To implement CRUD operations
- To build interactive UI using ReactJS
- To use NodeJS and ExpressJS for backend
- To store data in MongoDB
- To implement user interactions like likes

Technologies Used

- **HTML** – Structure
- **CSS** – Styling (Pink + Blue gradient UI)
- **JavaScript** – Logic
- **ReactJS** – Frontend
- **NodeJS** – Backend runtime
- **ExpressJS** – API handling
- **MongoDB** – Database
- **Axios** – API communication

Theory

A social media platform allows users to create and interact with posts.

Frontend (ReactJS)

Handles UI, post creation, and interactions.

Backend (NodeJS + ExpressJS)

Manages APIs for posts, likes, and deletion.

Database (MongoDB)

Stores post content and like count.

Styling (Different CSS Concept)

This version uses:

- Pink + Blue gradient background
- Card-based post layout
- Modern UI with rounded elements

Working

1. User writes a post
2. Post is saved in MongoDB
3. Posts are displayed
4. User can like or delete

Code:

1. App.jsx (Frontend)

```
import React, { useEffect, useState } from "react";
import axios from "axios";

function App() {
  const [content, setContent] = useState("");
  const [posts, setPosts] = useState([]);

  const fetchPosts = async () => {
    const res = await axios.get("http://localhost:5000/posts");
    setPosts(res.data);
  };

  useEffect(() => {
    { fetchPosts();
  }, []);

  const addPost = async () => {
    await axios.post("http://localhost:5000/add", { content });
    setContent("");
    fetchPosts();
  };

  const likePost = async (id) => {
    await axios.put(`http://localhost:5000/like/${id}`);
    fetchPosts();
  };

  const deletePost = async (id) => {
    await axios.delete(`http://localhost:5000/delete/${id}`);
    fetchPosts();
  };
}
```

```

return (
  <div
    style={{ minHeight:
      "100vh",
      background: "linear-gradient(to right, #ff9a9e, #89f7fe)",
      padding: "20px"
    }}>
    <h1 style={{ textAlign: "center" }}>Social Media</h1>

    <div
      style={{ background:
        "white", padding:
        "20px", borderRadius:
        "10px", maxWidth:
        "500px", margin:
        "auto"
      }}>
      <textarea
        placeholder="What's on your mind?"
        value={content}
        onChange={(e) => setContent(e.target.value)}
        style={{ width: "100%" }}
      ></textarea>

      <button onClick={addPost}
        style={{
          background: "#ff6b81",
          color: "white",
          marginTop: "10px"
        }}>
        Post
      </button>
    </div>

    <div style={{ marginTop: "20px" }}>
      {posts.map((post) => (
        <div key={post._id}
          style={{ backgroun
            d: "#fff",
            margin: "10px auto",
            padding: "15px",
            borderRadius: "10px",
            maxWidth: "500px"
          }}>
          <p>{post.content}</p>
          <p>Likes: {post.likes}</p>

          <button onClick={() => likePost(post._id)}
            style={{ background: "#74b9ff", color: "white", marginRight: "5px" }}>
            Like

```

</button>


```

        <button onClick={() => deletePost(post._id)}
          style={{ background: "#ff4757", color: "white" }}>
          Delete
        </button>
      </div>
    )}
  </div>
</div>
);
}

```

export default App;

2. server.js (Backend)

```

const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");

const app = express();

app.use(express.json());
app.use(cors());

// MongoDB connection
mongoose.connect("mongodb://127.0.0.1:27017/socialDB",
  { useNewUrlParser: true,
    useUnifiedTopology: true,
  })
.then(() => console.log("MongoDB Connected"))
.catch(err => console.log(err));

// Schema
const schema = new
  mongoose.Schema({ content: String,
    likes: { type: Number, default: 0 },
  });

const Post = mongoose.model("Post", schema);

// Add
app.post("/add", async (req, res) =>
  { const post = new Post(req.body);
    await post.save();
    res.send("Added");
  });

// Get
app.get("/posts", async (req, res) =>
  { const posts = await Post.find();
    res.json(posts);
  });

```

```
});

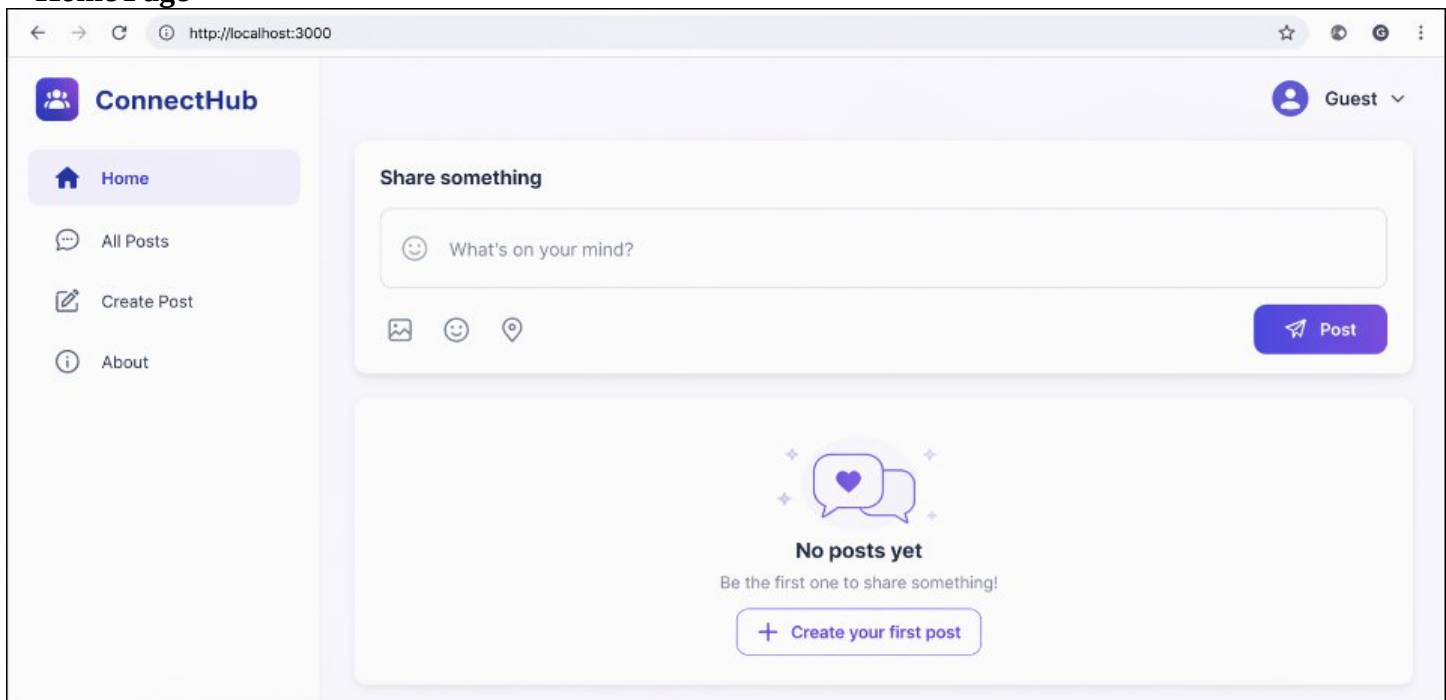
// Like
app.put("/like/:id", async (req, res) => {
  await Post.findByIdAndUpdate(req.params.id, { $inc: { likes: 1 } });
  res.send("Liked");
});

// Delete
app.delete("/delete/:id", async (req, res) => {
  { await
    Post.findByIdAndDelete(req.params.id);
    res.send("Deleted");
  });

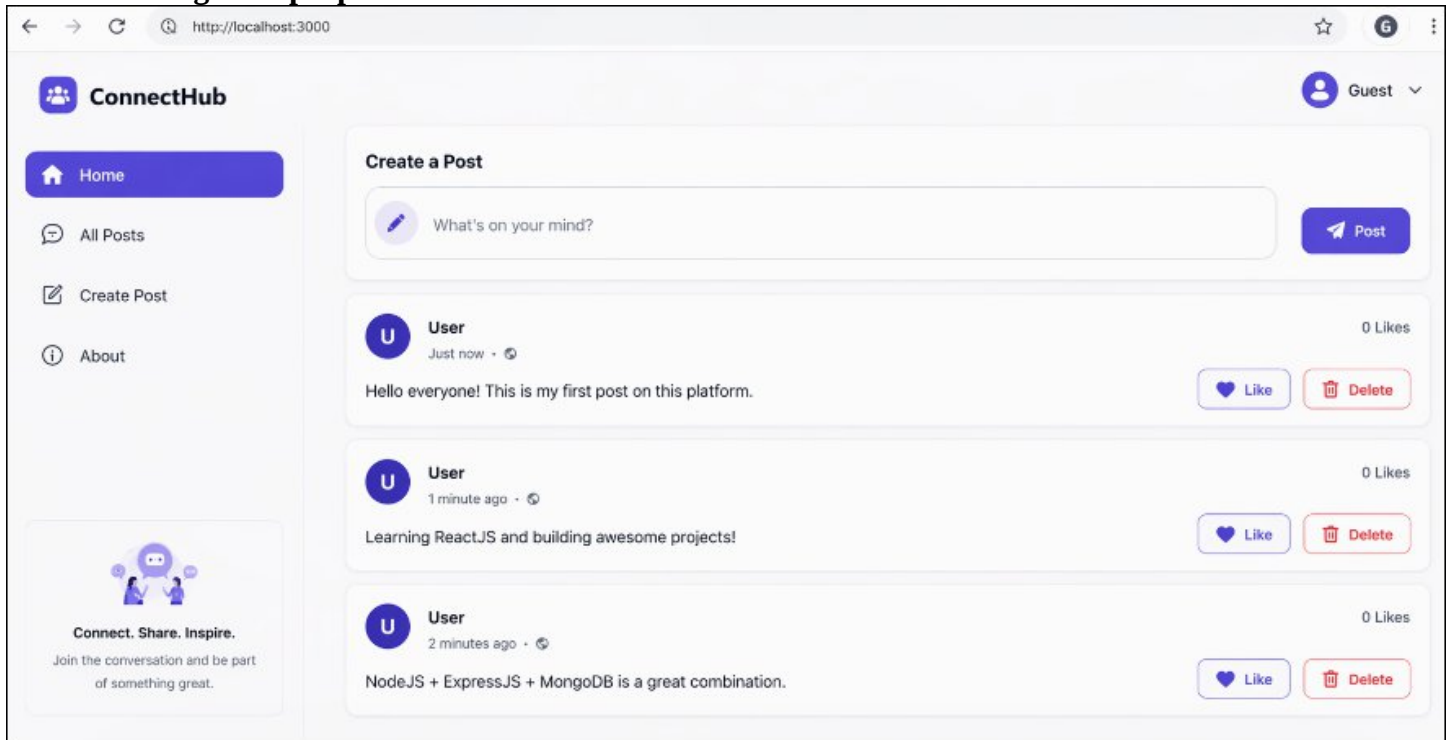
app.listen(5000, () => {
  console.log("Server running on http://localhost:5000");
});
```

Output:

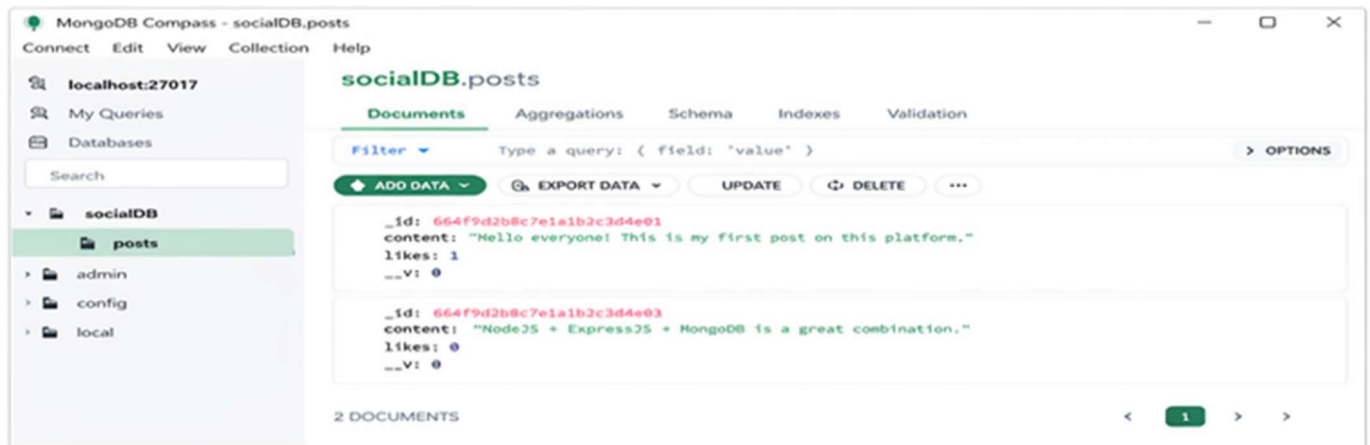
Home Page



After adding multiple posts



MongoDB Compass (Database)



Viva Questions

1. What is a social media platform?

A social media platform is a web application that allows users to create, share, and interact with content and connect with other users.

2. What is the role of MongoDB in this project?

MongoDB is used to store and manage data such as posts, user information, and likes in a flexible, document-based format.

3. What does the “like” feature do in backend?

The “like” feature updates the like count of a post in the database, typically by incrementing the value using the \$inc operator.

4. How does React communicate with backend?

React communicates with the backend using HTTP requests through libraries like Axios to send and receive data from APIs.

5. What is REST API?

A REST API is an architectural style that enables communication between frontend and backend using standard HTTP methods like GET, POST, PUT, and DELETE.

Experiment-4

Aim

To design and develop a weather forecasting web application that displays real-time weather data using ReactJS and REST API integration.

Objectives

- To understand API integration in web applications
- To fetch and display real-time weather data
- To build responsive UI using ReactJS
- To use JavaScript for handling API responses
- To design visually appealing UI using CSS
- To implement search-based weather functionality

Technologies Used

- **HTML** – Structure
- **CSS** – Styling (Modern gradient UI theme)
- **JavaScript** – Logic handling
- **ReactJS** – Frontend framework
- **REST API** – Fetch weather data (e.g., OpenWeather API)
- **Axios** – API requests

Theory

A weather forecasting app is a web application that retrieves and displays real-time weather information using external APIs.

1. Frontend (ReactJS)

ReactJS is used to build a dynamic UI where users can search for cities and view weather details like temperature, humidity, and conditions.

2. API Integration

The application uses a REST API to fetch live weather data. The API returns JSON data which is parsed and displayed.

3. Data Handling

JavaScript processes API responses and updates the UI dynamically.

4. Styling (Different CSS Concept)

This project uses:

- Gradient backgrounds
- Card-based UI
- Centered layout with modern design

Working of the System

1. User enters city name
2. React sends API request
3. Weather data is fetched
4. Data is displayed on screen
5. UI updates dynamically

Code:

1. App.jsx (Frontend)

```
import React, { useState } from "react";
import axios from "axios";
```

```
function App() {
  const [city, setCity] = useState("");
  const [weather, setWeather] = useState(null);
```

```
  const getWeather = async () =>
  { try {
    const res = await axios.get(
```

```
`https://api.openweathermap.org/data/2.5/weather?q=${city}&appid=YOUR_API_KEY&units=metric`
    );
    setWeather(res.data);
  } catch (error)
  { alert("City not
    found");
  }
};
```

```
return (
  <div
    style={{ height:
      "100vh",
    background: "linear-gradient(to right, #4facfe, #00f2fe)",
    display: "flex",
    justifyContent: "center",
    alignItems: "center"
  }}>
    <div
      style={{ background:
        "white", padding:
```

"30px", borderRadius:
"10px", textAlign:
"center"

```

}}>
<h1>Weather App</h1>

<input
  type="text"
  placeholder="Enter city"
  onChange={(e) => setCity(e.target.value)}
/>
<br /><br />

<button onClick={getWeather}>Get Weather</button>

{weather && (
  <div style={{ marginTop: "20px" }}>
    <h2>{weather.name}</h2>
    <p>Temperature: {weather.main.temp} °C</p>
    <p>Humidity: {weather.main.humidity}%</p>
    <p>Condition: {weather.weather[0].main}</p>
  </div>
)}
</div>
</div>
);
}

```

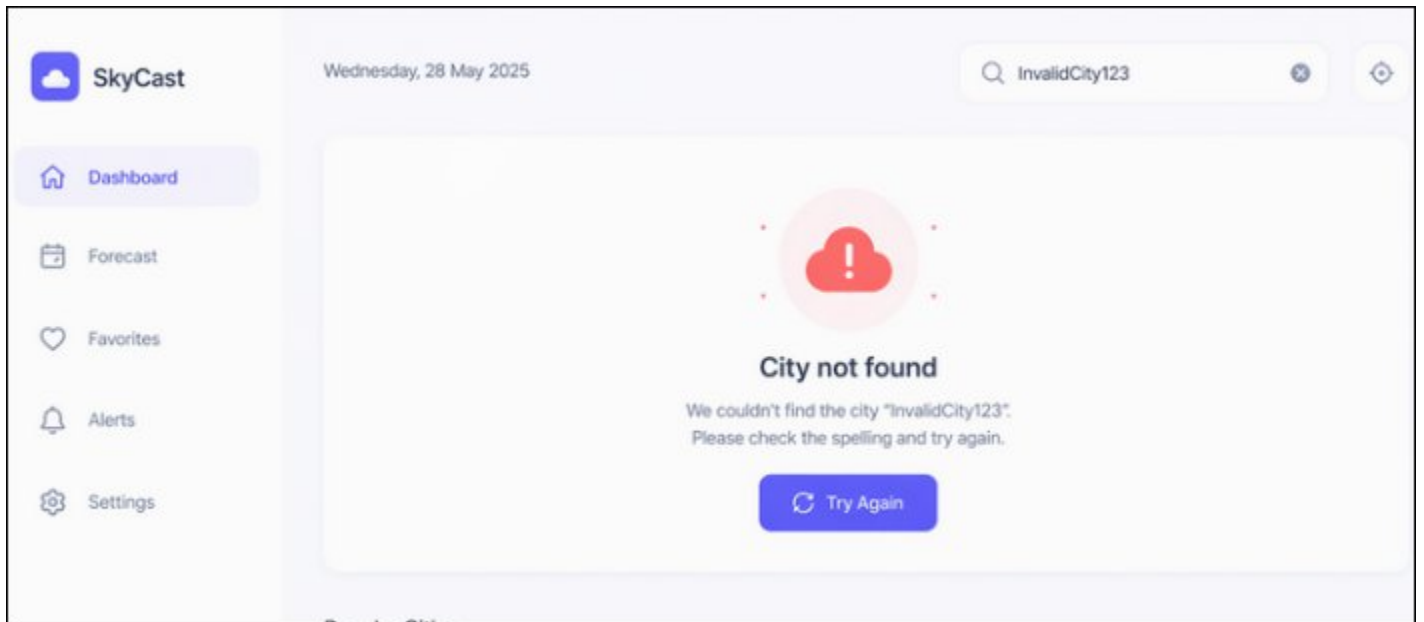
export default App;

Output:

After searching for a city



Error



Viva Questions

1. What is a REST API?

A REST API is a system that allows communication between a client and server using HTTP methods like GET, POST, PUT, and DELETE.

2. What does Axios do?

Axios is used to send HTTP requests to APIs and fetch or send data between the frontend and backend.

3. What data does weather API return?

A weather API returns data such as temperature, humidity, wind speed, and weather conditions (e.g., cloudy, rainy, sunny).

4. What happens if city is invalid?

If the city is invalid, an error occurs which is handled using mechanisms like try-catch or error responses from the API.

5. Why use ReactJS?

ReactJS is used to build fast, dynamic, and interactive user interfaces using reusable components.

Experiment-5

Aim

To design and develop a full-stack bookstore and library management system that allows adding, viewing, updating stock, and deleting books using modern web technologies.

Objectives

- To understand full-stack application development
- To manage inventory (books) using CRUD operations
- To build structured UI using ReactJS
- To develop REST APIs using NodeJS and ExpressJS
- To store book records in MongoDB
- To display data in tabular format

Technologies Used

- **HTML** – Structure
- **CSS** – Styling (Dashboard + Table UI style)
- **JavaScript** – Logic
- **ReactJS** – Frontend
- **NodeJS** – Backend runtime
- **ExpressJS** – API handling
- **MongoDB** – Database
- **Axios** – API communication

Theory

A bookstore library system is used to manage books and their stock in a digital format.

1. Frontend (ReactJS)

ReactJS is used to create a structured UI with forms and tables to display books and stock levels.

2. Backend (NodeJS + ExpressJS)

Handles API requests such as adding books, updating stock, and deleting records.

3. Database (MongoDB)

Stores book details like title, author, and quantity.

4. Styling (Different CSS Concept)

This project uses:

- Table-based UI
- Dashboard layout
- Clean admin panel design

Working of the System

1. User enters book details
2. Data is sent to backend
3. MongoDB stores book records
4. Books are displayed in table
5. User can update stock or delete

Code:

1. App.jsx (Frontend)

```
import React, { useEffect, useState } from "react";
import axios from "axios";

function App() {
  const [title, setTitle] = useState("");
  const [author, setAuthor] = useState("");
  const [quantity, setQuantity] = useState("");
  const [books, setBooks] = useState([]);

  // Fetch books
  const fetchBooks = async () => {
    const res = await axios.get("http://localhost:5000/books");
    setBooks(res.data);
  };

  useEffect(() => {
    { fetchBooks();
  }, []);

  // Add book
  const addBook = async () => {
    await axios.post("http://localhost:5000/add",
      { title,
        author,
        quantity,
      });
    setTitle("");
    setAuthor("");
    setQuantity("");
    fetchBooks();
  };
}
```

```

};

// Delete book
const deleteBook = async (id) => {
  await axios.delete(`http://localhost:5000/delete/${id}`);
  fetchBooks();
};

// Update stock
const updateStock = async (id) => {
  await axios.put(`http://localhost:5000/update/${id}`);
  fetchBooks();
};

return (
  <div style={{ padding: "20px" }}>
    <h1>Library Stock System</h1>

    <input
      placeholder="Book Title"
      value={title}
      onChange={(e) => setTitle(e.target.value)}
    />
    <input
      placeholder="Author"
      value={author}
      onChange={(e) => setAuthor(e.target.value)}
    />
    <input
      placeholder="Quantity"
      value={quantity}
      onChange={(e) => setQuantity(e.target.value)}
    />

    <button onClick={addBook}>Add Book</button>

    <hr />

    <table border="1" width="100%">
      <thead>
        <tr>
          <th>Title</th>
          <th>Author</th>
          <th>Stock</th>
          <th>Actions</th>
        </tr>
      </thead>

      <tbody>
        {books.map((book) => (
          <tr key={book._id}>

```

```

        <td>{book.title}</td>
        <td>{book.author}</td>
        <td>{book.quantity}</td>
        <td>
            <button onClick={() => updateStock(book._id)}>+1</button>
            <button onClick={() => deleteBook(book._id)}>Delete</button>
        </td>
    </tr>
    )}
</tbody>
</table>
</div>
);
}

```

export default App;

2. server.js (Backend)

```

const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");

const app = express();

app.use(express.json());
app.use(cors());

// MongoDB connection
mongoose.connect("mongodb://127.0.0.1:27017/libraryDB",
{ useNewUrlParser: true,
  useUnifiedTopology: true,
})
.then(() => console.log("MongoDB Connected"))
.catch(err => console.log(err));

// Schema
const bookSchema = new
  mongoose.Schema({ title: String,
    author: String,
    quantity: Number,
  });

const Book = mongoose.model("Book", bookSchema);

// Add Book
app.post("/add", async (req, res) =>
  { const book = new
    Book(req.body); await book.save();
    res.send("Book Added");
  });

```

```
// Get Books
app.get("/books", async (req, res) => {
  const books = await Book.find();
  res.json(books);
});

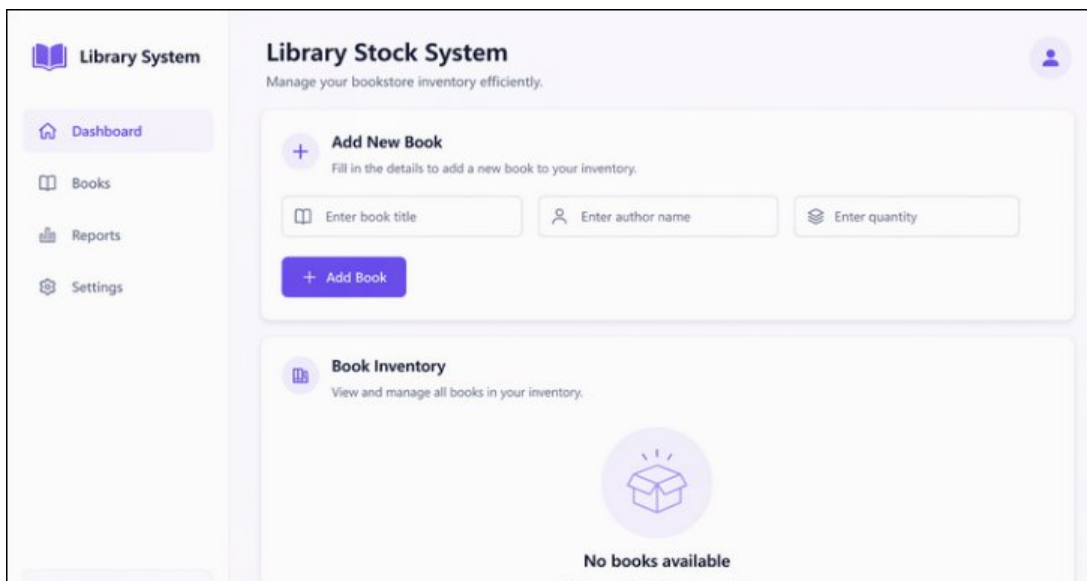
// Update Stock (+1)
app.put("/update/:id", async (req, res) => {
  await Book.findByIdAndUpdate(req.params.id, {
    $inc: { quantity: 1 },
  });
  res.send("Stock Updated");
});

// Delete Book
app.delete("/delete/:id", async (req, res) => {
  await
  Book.findByIdAndDelete(req.params.id);
  res.send("Deleted");
});

// Server
app.listen(5000, () => {
  console.log("Server running on http://localhost:5000");
});
```

Output:

Home Page



After adding multiple books

Library System

Dashboard

Books

Reports

Settings

Library Stock System

Manage your bookstore inventory efficiently.

+ Add New Book

Fill in the details to add a new book to your inventory.

Enter book title

Enter author name

Enter quantity

+ Add Book

Book Inventory

View and manage all books in your inventory.

Search books...

#	Book Title	Author	Quantity	Date Added	Actions
1	The Alchemist	Paulo Coelho	4	28 May 2025	
2	Atomic Habits	James Clear	6	27 May 2025	
3	Rich Dad Poor Dad	Robert Kiyosaki	3	27 May 2025	

After deleting a book

Library System

Dashboard

Books

Reports

Settings

Library Stock System

Manage your bookstore inventory efficiently.

+ Add New Book

Fill in the details to add a new book to your inventory.

Enter book title

Enter author name

Enter quantity

+ Add Book

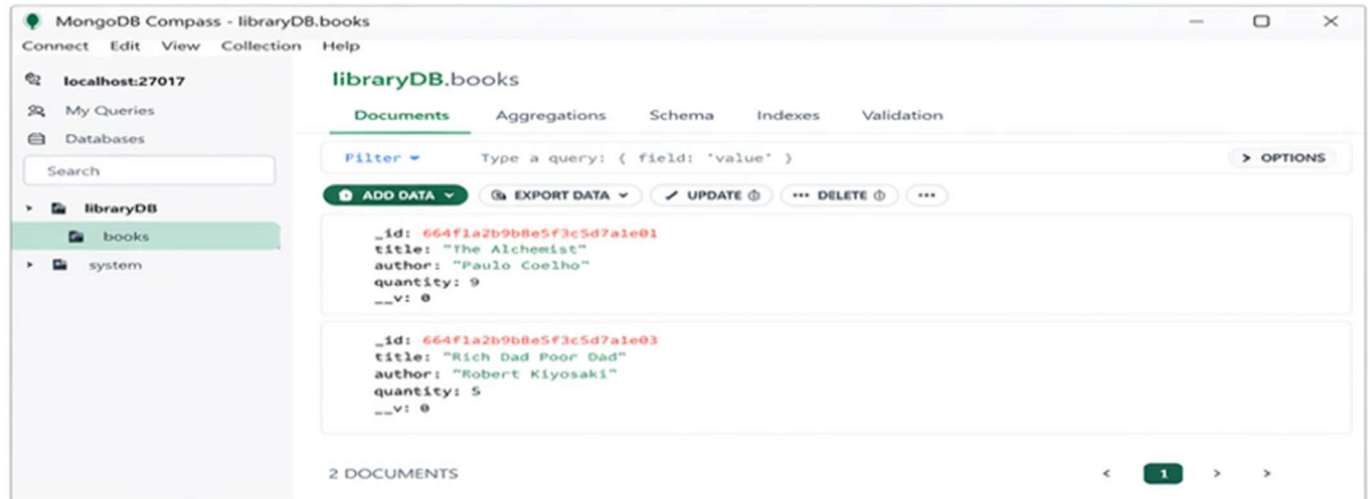
Book Inventory

View and manage all books in your inventory.

Search books...

#	Book Title	Author	Quantity	Date Added	Actions
1	The Alchemist	Paulo Coelho	4	28 May 2025	
2	Atomic Habits	James Clear	6	27 May 2025	
3	Rich Dad Poor Dad	Robert Kiyosaki	3	27 May 2025	

MongoDB Compass (Database)



Viva Questions

1. What is inventory management?

Inventory management is the process of tracking and managing stock items (such as books) to ensure proper availability and avoid overstocking or shortages.

2. What does \$inc do in MongoDB?

The \$inc operator in MongoDB is used to increment (increase or decrease) a numeric field value by a specified amount.

3. Why use table UI here?

A table UI is used to display data in a structured format with rows and columns, making it easy to read, organize, and manage large amounts of data.

4. What is CRUD?

CRUD stands for Create, Read, Update, and Delete, which are the basic operations performed on data in a database.

5. How does frontend communicate with backend?

The frontend communicates with the backend using HTTP requests (like GET, POST, PUT, DELETE) through tools like Axios.

Experiment-6

Aim

To develop a CRUD (Create, Read, Update, Delete) application using MongoDB, NodeJS, and ExpressJS with HTML forms and JavaScript.

Objectives

- To understand CRUD operations in web applications
- To use HTML forms for data input
- To implement backend using NodeJS and ExpressJS
- To store and manage data using MongoDB
- To perform update and delete operations
- To connect frontend with backend using JavaScript

Technologies Used

- **HTML** – Form structure
- **CSS** – Styling (Form-based clean UI)
- **JavaScript** – Client-side logic
- **NodeJS** – Backend runtime
- **ExpressJS** – API creation
- **MongoDB** – Database

Theory

A CRUD application allows users to perform basic operations on data: Create, Read, Update, and Delete.

1. Frontend (HTML + JavaScript)

HTML forms are used to collect user input. JavaScript handles form submission and displays data dynamically.

2. Backend (NodeJS + ExpressJS)

Handles requests such as inserting, updating, and deleting data from the database.

3. Database (MongoDB)

Stores records in document format.

4. Styling (Different CSS Concept)

This project uses:

- Centered form layout
- Minimal UI
- Input-focused design

Working of the System

1. User fills form
2. Data is sent to backend
3. MongoDB stores data
4. Records are displayed
5. User can update or delete

Code:

1. index.html (Frontend)

```
<!DOCTYPE html>
<html>
<head>
  <title>CRUD App</title>
</head>

<body style="text-align:center;">

  <h1>CRUD Application</h1>

  <input id="name" placeholder="Enter Name">
  <input id="age" placeholder="Enter Age">

  <button onclick="addOrUpdate()">Add</button>
  <button onclick="clearForm()">Clear</button>

  <h2>All Records</h2>
  <div id="list"></div>

  <script src="script.js"></script>
</body>
</html>
```

2. script.js (Frontend Logic)

```
const API = "http://localhost:5000";
```

```
let editId = null;
```

```
// Fetch Data
```

```

async function fetchData() {
  const res = await fetch(`${API}/data`);
  const data = await res.json();

  document.getElementById("list").innerHTML = data.map(d => `
    <div style="margin:10px;">
      ${d.name} - ${d.age}
      <button onclick="editData('${d._id}','${d.name}','${d.age}')">Update</button>
      <button onclick="deleteData('${d._id}')">Delete</button>
    </div>
  `).join("");
}

```

```

// Add OR Update
async function addOrUpdate() {
  const name = document.getElementById("name").value;
  const age = document.getElementById("age").value;

  if (editId) {
    await fetch(`${API}/update/${editId}`,
      { method: "PUT",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ name, age })
      });
    editId = null;
  } else {
    await fetch(`${API}/add`,
      { method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ name, age })
      });
  }

  clearForm();
  fetchData();
}

```

```

// Set data for editing
function editData(id, name, age)
{ document.getElementById("name").value =
  name; document.getElementById("age").value =
  age; editId = id;
}

```

```

// Delete
async function deleteData(id) {
  await fetch(`${API}/delete/${id}`, { method: "DELETE" });
  fetchData();
}

```

```

// Clear form
Sameer Sharma

```

02314802723

```
function clearForm()
{ document.getElementById("name").value =
  ""; document.getElementById("age").value =
  ""; editId = null;
}
```

```
fetchData();
```

3. server.js (Backend)

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");
```

```
const app = express();
```

```
app.use(express.json());
app.use(cors());
```

```
// MongoDB connection
mongoose.connect("mongodb://127.0.0.1:27017/crudDB",
{ useNewUrlParser: true,
  useUnifiedTopology: true,
})
.then(() => console.log("MongoDB Connected"))
.catch(err => console.log(err));
```

```
// Schema
const schema = new
  mongoose.Schema({ name: String,
    age: Number,
  });
```

```
const Model = mongoose.model("Data", schema);
```

```
// CREATE
app.post("/add", async (req, res) =>
{ const data = new
  Model(req.body); await data.save();
  res.send("Added");
});
```

```
// READ
app.get("/data", async (req, res) =>
{ const data = await Model.find();
  res.json(data);
});
```

```
// UPDATE
app.put("/update/:id", async (req, res) => {
  await Model.findByIdAndUpdate(req.params.id, req.body);
  res.send("Updated");
});
```

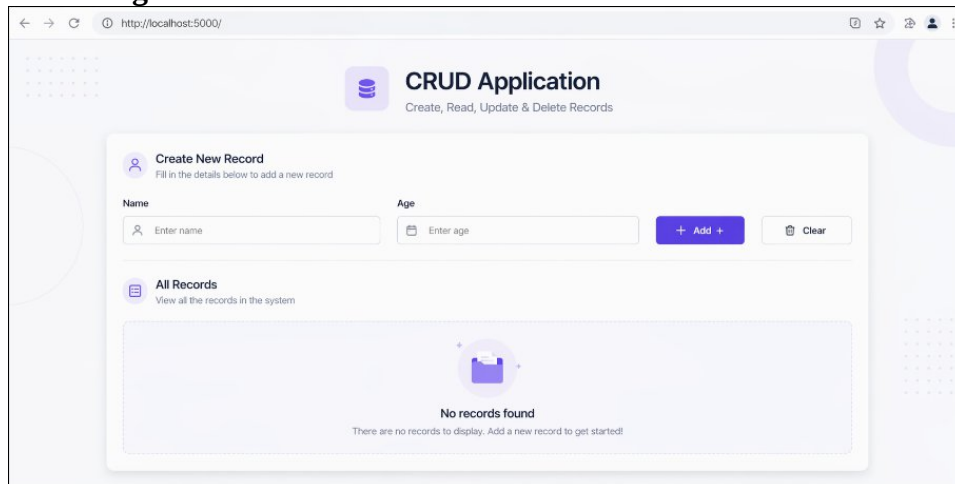
```
});

// DELETE
app.delete("/delete/:id", async (req, res) => {
  await
  Model.findByIdAndDelete(req.params.id);
  res.send("Deleted");
});

// Server
app.listen(5000, () => {
  console.log("Server running on http://localhost:5000");
});
```

Output:

Home Page



CRUD Application
Create, Read, Update & Delete Records

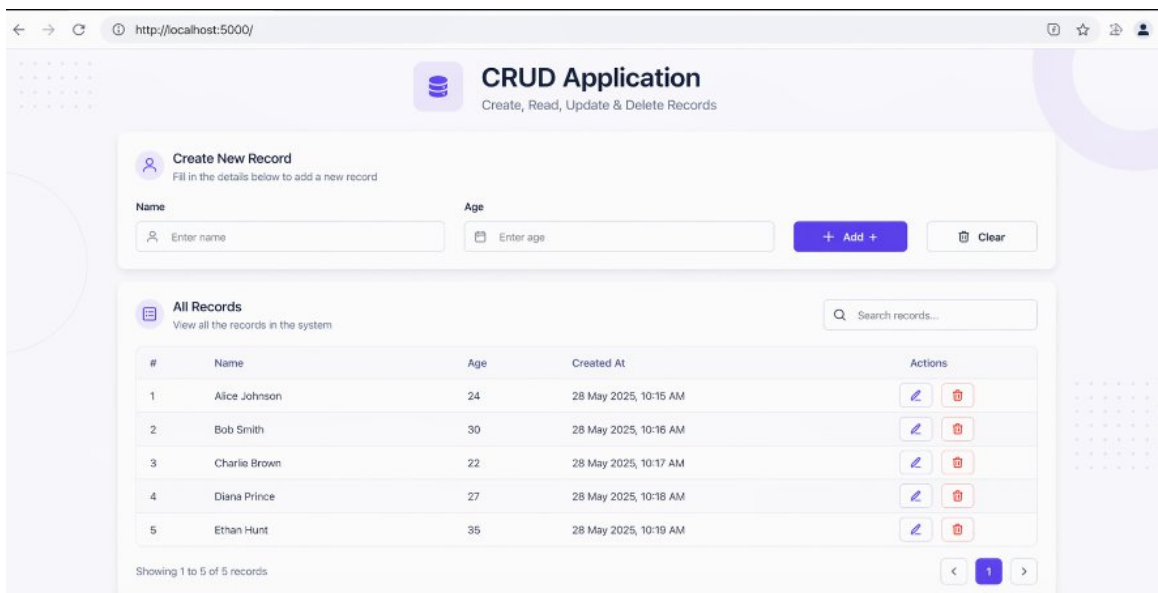
Create New Record
Fill in the details below to add a new record

Name: Age: + Add + Clear

All Records
View all the records in the system

No records found
There are no records to display. Add a new record to get started!

After adding multiple records



CRUD Application
Create, Read, Update & Delete Records

Create New Record
Fill in the details below to add a new record

Name: Age: + Add + Clear

All Records
View all the records in the system

Search records...

#	Name	Age	Created At	Actions
1	Alice Johnson	24	28 May 2025, 10:15 AM	Edit Delete
2	Bob Smith	30	28 May 2025, 10:16 AM	Edit Delete
3	Charlie Brown	22	28 May 2025, 10:17 AM	Edit Delete
4	Diana Prince	27	28 May 2025, 10:18 AM	Edit Delete
5	Ethan Hunt	35	28 May 2025, 10:19 AM	Edit Delete

Showing 1 to 5 of 5 records

< 1 >

After updating a record (Bob Smith age -> 31)

The screenshot shows the 'Update Record' form in the CRUD Application. The form has two input fields: 'Name' with the value 'Alice Johnson' and 'Age' with the value '24'. Below the inputs are 'Cancel' and 'Update Record' buttons. A 'Record Information' section at the bottom shows 'Created At: 28 May 2025, 10:15 AM'.

After deleting a record

The screenshot shows the 'All Records' page in the CRUD Application. A green message bar at the top says 'Record deleted successfully.' Below it is a table with 4 records. The table has columns: #, Name, Age, Created At, and Actions. The records are: 1. Alice Johnson (Age 24), 2. Charlie Brown (Age 22), 3. Diane Prince (Age 27), and 4. Ethan Hunt (Age 35). The 'Actions' column contains edit and delete icons for each record. At the bottom, it says 'Showing 1 to 4 of 4 records'.

#	Name	Age	Created At	Actions
1	Alice Johnson	24	28 May 2025, 10:15 AM	[Edit] [Delete]
2	Charlie Brown	22	28 May 2025, 10:17 AM	[Edit] [Delete]
3	Diane Prince	27	28 May 2025, 10:18 AM	[Edit] [Delete]
4	Ethan Hunt	35	28 May 2025, 10:19 AM	[Edit] [Delete]

MongoDB Compass (Database)

The screenshot shows the MongoDB Compass interface for the 'crudDB.people' collection. The 'Documents' tab is active, showing a list of documents. The first document is: `{_id: '664f9a9f1e3b2c4d8e1a1a11', name: 'Alice Johnson', age: 25}`. The second document is: `{_id: '664f9a9f1e3b2c4d8e1a1a12', name: 'Bob Smith', age: 31}`. The interface includes a sidebar with the database structure, a top menu, and a bottom status bar showing '2 DOCUMENTS'.

Viva Questions

1. What is CRUD?

CRUD stands for Create, Read, Update, and Delete, the four basic operations used to manage data in a database.

2. What is the role of HTML forms?

HTML forms are used to collect user input, such as text or other data, and submit it to the server for processing.

3. What does ExpressJS do?

Express.js is a backend framework used to create APIs, handle routes, process requests, and manage server-side logic.

4. How is data stored in MongoDB?

MongoDB stores data as JSON-like documents (BSON format) inside collections.

5. What is the role of JavaScript here?

JavaScript is used to send requests, handle responses, and update the user interface dynamically.

Experiment-7

Aim

To design and develop an inventory management platform that allows users to manage products, track stock levels, and perform CRUD operations using modern web technologies.

Objectives

- To understand inventory management systems
- To implement CRUD operations on products
- To build interactive UI using ReactJS
- To develop backend APIs using NodeJS and ExpressJS
- To store inventory data in MongoDB
- To visually represent stock levels

Technologies Used

- **ReactJS** – Frontend
- **NodeJS** – Backend runtime
- **ExpressJS** – API handling
- **MongoDB** – Database
- **CSS** – Styling (Card-based UI + colored stock indicators)
- **Axios** – API communication

Theory

An inventory management system helps in tracking and managing stock of products efficiently.

1. Frontend (ReactJS)

React is used to display products in card format with stock status and actions.

2. Backend (NodeJS + ExpressJS)

Handles API requests like adding products, updating stock, and deleting items.

3. Database (MongoDB)

Stores product details like name, price, and quantity.

4. Styling (Different CSS Concept)

This project uses:

- Card layout

- Color indicators:
 - Green In Stock
 - Yellow Low Stock
 - Red Out of Stock

Working of the System

1. User adds product
2. Data is stored in MongoDB
3. Products are displayed as cards
4. Stock is updated dynamically
5. User can delete items

Code:

1. App.jsx (Frontend)

```
import React, { useEffect, useState } from "react";
import axios from "axios";

function App() {
  const [name, setName] = useState("");
  const [price, setPrice] = useState("");
  const [quantity, setQuantity] = useState("");
  const [items, setItems] = useState([]);

  const fetchItems = async () => {
    const res = await axios.get("http://localhost:5000/items");
    setItems(res.data);
  };

  useEffect(() => {
    { fetchItems();
  }, []);

  const addItem = async () => {
    await axios.post("http://localhost:5000/add",
      { name,
        price,
        quantity,
      });
    setName("");
    setPrice("");
    setQuantity("");
    fetchItems();
  };

  const updateStock = async (id, type) => {
    await axios.put(`http://localhost:5000/update/${id}`, { type });
  };
}
```

```

    fetchItems();
  };

const deleteItem = async (id) => {
  await axios.delete(`http://localhost:5000/delete/${id}`);
  fetchItems();
};

const getStockColor = (qty) =>
{ if (qty === 0) return "red";
  if (qty < 5) return "orange";
  return "green";
};

return (
  <div style={{ padding: "20px" }}>
    <h1>Inventory Management</h1>

    <input placeholder="Product Name" value={name}
      onChange={(e) => setName(e.target.value)} />
    <input placeholder="Price" value={price}
      onChange={(e) => setPrice(e.target.value)} />
    <input placeholder="Quantity" value={quantity}
      onChange={(e) => setQuantity(e.target.value)} />

    <button onClick={addItem}>Add Product</button>

    <hr />

    <div style={{ display: "flex", flexWrap: "wrap" }}>
      {items.map((item) => (
        <div key={item._id}
          style={{
            border: "1px solid #ccc",
            padding: "15px",
            margin: "10px",
            width: "200px",
            borderRadius: "10px"
          }}>

          <h3>{item.name}</h3>
          <p>Price: ₹{item.price}</p>

          <p style={{ color: getStockColor(item.quantity) }}>
            Stock: {item.quantity}
          </p>

          <button onClick={() => updateStock(item._id, "inc")}>+</button>
          <button onClick={() => updateStock(item._id, "dec")}>-</button>
          <button onClick={() => deleteItem(item._id)}>Delete</button>

```

</div>

```

    })}
  </div>
</div>
);
}

```

```
export default App;
```

2. server.js (Backend)

```

const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");

```

```
const app = express();
```

```

app.use(express.json());
app.use(cors());

```

```

// MongoDB connection
mongoose.connect("mongodb://127.0.0.1:27017/inventoryDB",
{ useNewUrlParser: true,
  useUnifiedTopology: true,
})
.then(() => console.log("MongoDB Connected"))
.catch(err => console.log(err));

```

```

// Schema
const itemSchema = new
  mongoose.Schema({ name: String,
    price: Number,
    quantity: Number,
  });

```

```
const Item = mongoose.model("Item", itemSchema);
```

```

// Add item
app.post("/add", async (req, res) =>
  { const item = new Item(req.body);
    await item.save();
    res.send("Added");
  });

```

```

// Get items
app.get("/items", async (req, res) =>
  { const items = await Item.find();
    res.json(items);
  });

```

```
// Update stock
```

Sameer Sharma

02314802723

```

app.put("/update/:id", async (req, res) =>
  { const { type } = req.body;

  if (type === "inc") {
    await Item.findByIdAndUpdate(req.params.id, { $inc: { quantity: 1 } });
  } else {
    await Item.findByIdAndUpdate(req.params.id, { $inc: { quantity: -1 } });
  }

  res.send("Updated");
});

// Delete item
app.delete("/delete/:id", async (req, res) =>
  { await
    Item.findByIdAndDelete(req.params.id);
    res.send("Deleted");
  });
app.listen(5000, () => {
  console.log("Server running on http://localhost:5000");
});

```

Output(screenshots):

Home page

Adding multiple products

Inventory Management

Manage your library products and stock efficiently.

+ Add New Product

Fill in the details below to add a new product to inventory.

Name

Price (₹)

Quantity

Enter product name

₹ Enter price

Enter quantity

+ Add Product

Product Inventory

View and manage all products in your inventory.

Search products...

#	Product Name	Price (₹)	Quantity	Total Value (₹)	Actions
1	The Alchemist	250.00	15	3,750.00	
2	Atomic Habits	499.00	8	3,992.00	

MongoDB compass (Database)

MongoDB Compass - inventoryDB.items

Connect Edit View Collection Help

localhost:27017

My Queries

Databases

Search

InventoryDB

items

admin

config

local

inventoryDB.items

Documents Aggregations Schema Indexes Validation

Filter Type a query: { field: 'value' } > OPTIONS

ADD DATA

EXPORT DATA

UPDATE

DELETE

_id: 664f3b9a4b8e1f2b3c4d5e01

name: "Laptop"

price: 55000

quantity: 11

--v: 0

_id: 664f3b9a4b8e1f2b3c4d5e02

name: "Smartphone"

price: 18000

quantity: 1

--v: 0

_id: 664f3b9a4b8e1f2b3c4d5e04

name: "Keyboard"

price: 1000

quantity: 6

--v: 0

3 DOCUMENTS

< 1 >

Viva Questions

1. What is inventory management?

Inventory management is the process of managing and tracking the stock of products to ensure proper availability and control.

2. What does \$inc do?

The \$inc operator increases or decreases the value of a numeric field by a specified amount.

3. Why use card UI?

Card UI is used for better visualization and organized display of items, making product information easy to view and interact with.

4. What is MongoDB used for?

MongoDB is used to store and manage product data in a document-based format.

5. How is stock updated?

Stock is updated by sending a PUT request to the backend, where the value is modified in the database using the \$inc operator.

Experiment-8

Aim

To design and develop an appointment booking system that allows users to schedule, view, and cancel appointments using modern web technologies.

Objectives

- To understand scheduling systems
- To implement CRUD operations for appointments
- To build interactive UI using ReactJS
- To develop backend APIs using NodeJS and ExpressJS
- To store appointment data in MongoDB
- To manage time slots and booking status

Technologies Used

- **HTML** – Structure
- **CSS** – Styling (Red + Grey theme UI)
- **ReactJS** – Frontend
- **NodeJS** – Backend runtime
- **ExpressJS** – API handling
- **MongoDB** – Database
- **Axios** – API communication

Theory

An appointment booking system allows users to reserve time slots for services.

1. Frontend (ReactJS)

Provides UI for selecting date, time, and entering user details.

2. Backend (NodeJS + ExpressJS)

Handles booking, cancellation, and fetching appointments.

3. Database (MongoDB)

Stores appointment details like name, date, and time.

4. Styling (Different CSS Concept)

This project uses:

- Red theme for actions (booking, cancel)
- Grey cards for appointments
- Professional scheduling UI

Working of the System

1. User enters name, date, and time
2. Appointment is saved in MongoDB
3. Appointments are displayed
4. User can cancel booking

Code:

1. App.jsx (Frontend)

```
import React, { useEffect, useState } from "react";
import axios from "axios";

function App() {
  const [name, setName] = useState("");
  const [date, setDate] = useState("");
  const [time, setTime] = useState("");
  const [appointments, setAppointments] = useState([]);

  const fetchAppointments = async () => {
    const res = await axios.get("http://localhost:5000/appointments");
    setAppointments(res.data);
  };

  useEffect(() => {
    { fetchAppointments();
    }, []);

  const bookAppointment = async () => {
    await axios.post("http://localhost:5000/add",
      { name,
        date,
        time,
      });

    setName("");
    setDate("");
    setTime("");
    fetchAppointments();
  };

  const cancelAppointment = async (id) => {
    await axios.delete(`http://localhost:5000/delete/${id}`);
    fetchAppointments();
  };
}
```

```

};

return (
  <div
    style={{ background:
      "#f5f5f5", minHeight:
      "100vh", padding:
      "20px"
    }}>
    <h1 style={{ color: "#c0392b" }}>Appointment Booking</h1>

    <div
      style={{ background:
        "#fff", padding:
        "20px", borderRadius:
        "10px",
        marginBottom: "20px"
      }}>
      <input placeholder="Name" value={name}
        onChange={(e) => setName(e.target.value)} />
      <input type="date" value={date}
        onChange={(e) => setDate(e.target.value)} />
      <input type="time" value={time}
        onChange={(e) => setTime(e.target.value)} />

      <button
        style={{ background:
          "#e74c3c", color:
          "white", marginLeft:
          "10px"
        }} onClick={bookAppointment}>
        Book
      </button>
    </div>

    <div>
      {appointments.map((a) => (
        <div key={a._id}
          style={{
            background: "#ecf0f1",
            padding: "15px",
            marginBottom: "10px",
            borderRadius: "8px"
          }}>
          <h3>{a.name}</h3>
          <p>{a.date} | {a.time}</p>

          <button
            style={{ background:
              "#c0392b", color:
              "white"
            }}

```

```
}}  
  onClick={() => cancelAppointment(a._id)}>  
    Cancel  
</button>
```

```

        </div>
      )}
    </div>
  </div>
);
}

```

export default App;

2. server.js (Backend)

```

const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");

const app = express();

app.use(express.json());
app.use(cors());

// MongoDB connection
mongoose.connect("mongodb://127.0.0.1:27017/appointmentDB",
{ useNewUrlParser: true,
  useUnifiedTopology: true,
})
.then(() => console.log("MongoDB Connected"))
.catch(err => console.log(err));

// Schema
const schema = new
  mongoose.Schema({ name: String,
    date: String,
    time: String,
  });

const Appointment = mongoose.model("Appointment", schema);

// Add appointment
app.post("/add", async (req, res) =>
  { const a = new Appointment(req.body);
    await a.save();
    res.send("Booked");
  });

// Get appointments
app.get("/appointments", async (req, res) =>
  { const data = await Appointment.find();
    res.json(data);
  });

```

```
// Delete appointment
app.delete("/delete/:id", async (req, res) => {
  await Appointment.findByIdAndDelete(req.params.id);
  res.send("Cancelled");
});
app.listen(5000, () => {
  console.log("Server running on http://localhost:5000");
});
```

Output(Screenshots):

Home page

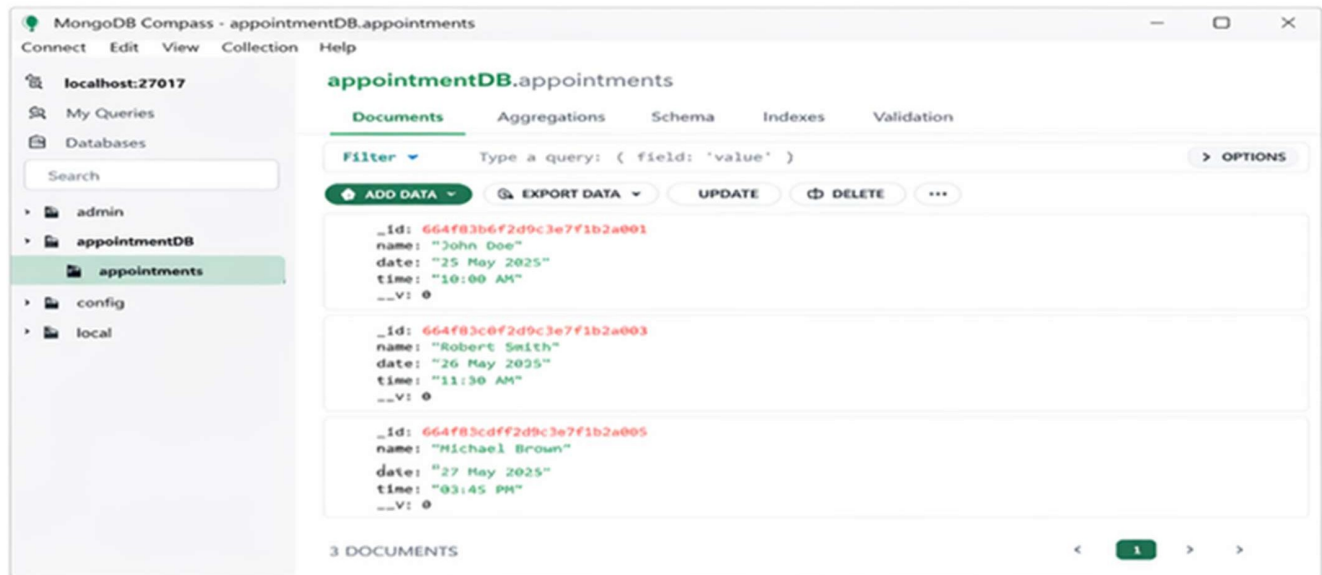
The screenshot shows the 'Scheduler' application's 'Dashboard' page. The left sidebar contains links for 'Dashboard', 'Appointments', 'Add Appointment', and 'About'. The main content area is titled 'Dashboard' with the subtitle 'Book your appointment with ease.' Below this is a 'Book New Appointment' form with three input fields: 'FULL NAME' (with placeholder 'e.g. Jane Doe'), 'SELECT DATE' (with placeholder 'mm/dd/yyyy' and a calendar icon), and 'SELECT TIME' (with placeholder '09:00 AM' and a dropdown arrow). A 'Book Appointment' button is to the right of the form. Below the form, there is a large light blue box with a calendar icon and the text 'No appointments yet. Book your first appointment using the form above! Your scheduled sessions will appear here in a clean, manageable list.'

After booking multiple Appointments

The screenshot shows the 'Scheduler' application's 'Dashboard' page after booking multiple appointments. The left sidebar is the same. The main content area is titled 'Dashboard' with the subtitle 'Book your appointment with ease.' Below this is the 'Book New Appointment' form. Below the form, there is a section titled 'Upcoming Appointments' with a 'View all →' link. This section contains a list of two appointments:

NAME	DATE & TIME	STATUS	ACTIONS
Sarah Jenkins General Consultation	Oct 24, 2023 • 10:00 AM	Confirmed	
Michael Chen Follow-up	Oct 25, 2023 • 02:30 PM	Pending	

MongoDB Compass (Database)



Viva Questions

1. What is an appointment system?

An appointment system is a system used to schedule and manage time slots for bookings or meetings.

2. What data is stored?

Data such as patient/user name, appointment date, and appointment time is stored in the database.

3. What does DELETE do?

The DELETE operation removes data from the database, such as cancelling an appointment.

4. Why use ReactJS?

React is used to create a dynamic and interactive user interface using reusable components.

5. What is MongoDB used for?

MongoDB is used to store and manage appointment data in the database.